

# Smart Expense Intelligence System

Full-Stack Financial Analytics • Project Completion Checklist & Technical Specification Report

## Executive Summary

The **Smart Expense Intelligence System** is a full-stack financial analytics web application built with Python 3.11+, FastAPI, Streamlit, SQLite, and Pytest. The system ingests bank statements (CSV/PDF), cleans raw narration strings, categorizes transactions via LLM with merchant-level caching, detects recurring subscription payments using a deterministic algorithm, and predicts cash-flow shortfalls using statistical linear projections.

## Complete Build Checklist

| Category      | Component / Feature        | Implementation Details                                               | Status    |
|---------------|----------------------------|----------------------------------------------------------------------|-----------|
| Ingestion     | CSV & PDF Parser           | Flexible header normalization for Indian bank statements             | COMPLETED |
| Ingestion     | Merchant Name Cleaner      | Regex pattern cascade stripping UPI, POS, IFSC junk tokens           | COMPLETED |
| AI Layer      | Batched LLM Categorization | 15 txns/request batching with strict JSON schema validation          | COMPLETED |
| AI Layer      | Merchant Category Cache    | SQLite cache so unique merchants trigger at most 1 API call          | COMPLETED |
| AI Layer      | Rule-based Fallback        | Zero-config keyword matching when no API key is set                  | COMPLETED |
| Analytics     | Recurring Subscriptions    | Pure deterministic algorithm (5% amount variance, interval patterns) | COMPLETED |
| Analytics     | Cash-Flow Forecasting      | 30-day rolling average spend rate forward projection                 | COMPLETED |
| Storage & API | SQLite Persistence         | Tables for transactions, category cache, and manual overrides        | COMPLETED |
| Storage & API | FastAPI REST API           | Endpoints for ingest, categorize, recurring, forecast, override      | COMPLETED |
| Frontend      | Streamlit Dashboard        | KPI cards, Plotly charts, shortfall alert banner, tables             | COMPLETED |
| Frontend      | Category Override UI       | Interactive dropdown allowing user correction with cache update      | COMPLETED |
| Data & Tests  | Synthetic Statement Data   | 62 transactions over 2 months, 6 subscriptions, electronics spike    | COMPLETED |
| Data & Tests  | Pytest Unit Test Suite     | 6 unit test cases covering monthly, weekly, irregular, edge cases    | COMPLETED |
| Extensions    | Docker Containerization    | Optional deployment container setup                                  | OPTIONAL  |
| Extensions    | Multi-Currency Support     | Optional support for \$, €, £                                        | OPTIONAL  |

## Architecture & Technical Trade-offs

### Key Interview Defense Talking Points:

- **Why AI for Categorization?** Merchant narrations are messy and unpredictable across banks. An LLM generalizes naturally. Cost is controlled via 15-txn batching and merchant-level SQLite caching.
- **Why Pure Algorithm for Recurring Payments?** Recurring detection is a constrained math problem ('similar amount at regular interval'). A pure algorithm is faster, free, 100% deterministic, and trivially unit-testable (6/6 tests passing).
- **Why Simple Statistics for Forecasting?** A 30-day rolling average linear projection is explainable, auditable, and stable without ML black-box risk.

## Verification Summary

**Unit Tests:** 6 passed in 0.24s (Pytest)

**FastAPI Backend:** Running on http://127.0.0.1:8000

**Streamlit Frontend:** Running on http://localhost:8501

**Detected Subscriptions:** Rent (₹32,000/mo), Cult Fit (₹999/mo), ACT Broadband (₹799/mo), Airtel (₹599/mo), Netflix (₹199/mo), Spotify (₹119/mo)

**Forecast Result:** will\_run\_short: True (Shortfall Date: 2025-08-01, Amount: ₹36,729.60)